

Habit Shaper - Technical Solution

Overview

Habit Shaper is a lightweight, web-based application that helps users build positive habits and break negative ones through daily tracking and streak management. This document outlines the complete technical solution covering system architecture, database design, API contracts, frontend design, deployment strategy, and testing approach.

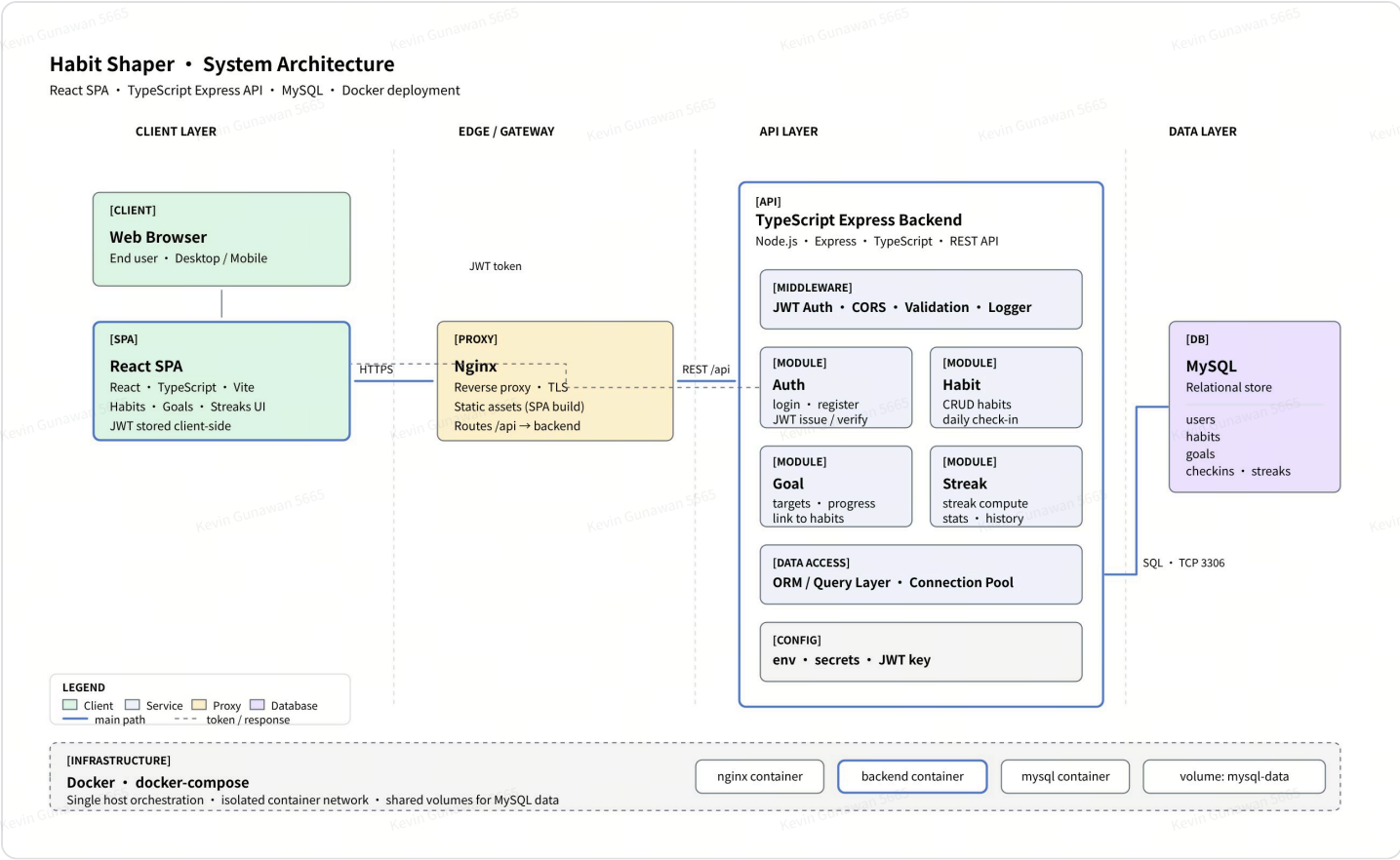


Key Technical Decisions:

- **Frontend:** React (TypeScript) with modern hooks and context
- **Backend:** Node.js + Express (TypeScript) REST API
- **Database:** MySQL 8.x with relational schema
- **Deployment:** Docker + docker-compose for single-host orchestration
- **Auth:** JWT-based stateless authentication

System Architecture

The system follows a **three-tier architecture** with clear separation of concerns between presentation, business logic, and data layers, all orchestrated via Docker containers.



Component Breakdown

Component	Technology	Responsibility
Web Client	React 18 + TypeScript + Vite	SPA handling UI rendering, form inputs, state management, and API communication
Reverse Proxy	Nginx	TLS termination, static asset serving (SPA build), routing /api requests to backend
API Server	Node.js + Express + TypeScript	Business logic, authentication, input validation, habit/goal/streak management
Database	MySQL 8.x	Persistent relational storage for users, habits, goals, logs, and streaks
Infrastructure	Docker + docker-compose	Container orchestration, isolated network, volume persistence for MySQL data

Database Design

Entity Relationship Diagram

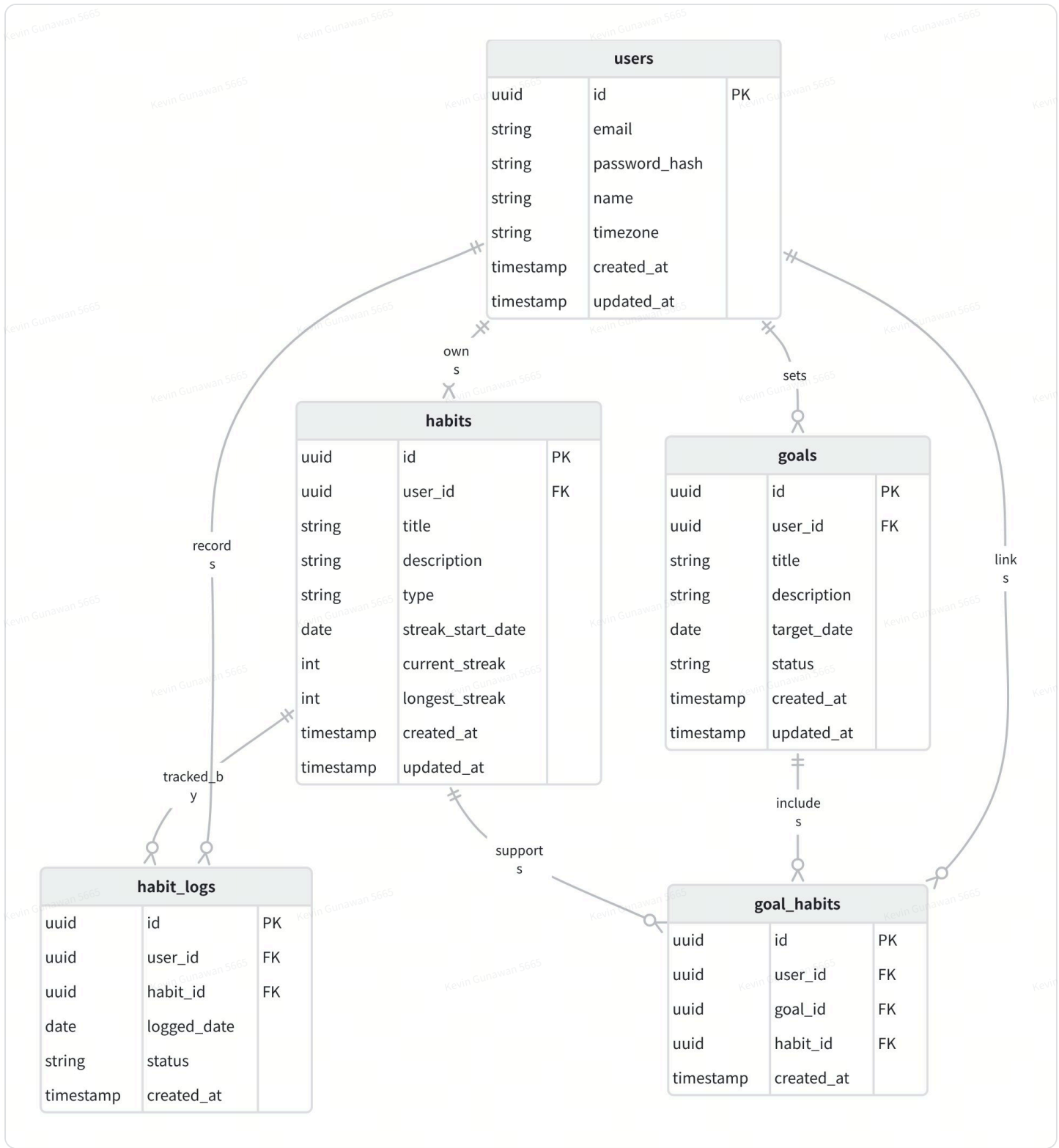


Table Definitions

users

Column	Type	Nullabl e	Constraints	Description
id	BIGINT	NO	PK, AUTO_INCREMENT	Unique user identifier

email	VARCHAR(255)	NO	UNIQUE INDEX	User email for authentication
password_hash	VARCHAR(255)	NO	-	bcrypt hashed password (12 rounds)
name	VARCHAR(100)	NO	-	Display name
timezone	VARCHAR(50)	NO	DEFAULT "UTC"	IANA home timezone (e.g., Asia/Jakarta). Used to determine "today" for streak calculations.
created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	Account creation time
updated_at	TIMESTAMP	YES	ON UPDATE CURRENT_TIMESTAMP	Last modification time

habits

Column	Type	Nullable	Constraints	Description
id	BIGINT	NO	PK, AUTO_INCREMENT	Unique habit identifier
user_id	BIGINT	NO	FK → users.id, INDEX	Owner of the habit
title	VARCHAR(200)	NO	-	Habit name
description	TEXT	YES	-	Optional detailed description
type	ENUM("BUILDING", "BREAKING")	NO	-	Whether building a new habit or breaking a bad one
streak_start_date	DATE	YES	-	For BREAKING: day after last relapse or creation date. Streak = daysBetween(streak_start_date, today)

current_streak	INT	NO	DEFAULT 0	Cached current streak (updated on mark/relapse)
longest_streak	INT	NO	DEFAULT 0	Best streak ever achieved
created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	NO	ON UPDATE CURRENT_TIMESTAMP	Last modification time

habit_logs

Column	Type	Nullable	Constraints	Description
id	BIGINT	NO	PK, AUTO_INCREMENT	Unique log entry identifier
user_id	BIGINT	NO	FK → users.id, INDEX	Owner of the log entry
habit_id	BIGINT	NO	FK → habits.id, INDEX	Associated habit
logged_date	DATE	NO	UNIQUE(habit_id, logged_date)	Date of the action in user's home timezone (one per day)
status	ENUM("COMPLETED", "RELAPSED")	NO	-	COMPLETED = building done, RELAPSED = breaking failed. MISSED/CLEAN derived at read time.
created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	UTC timestamp of when log was created

goals

Column	Type	Nullable	Constraints	Description
id	BIGINT	NO	PK, AUTO_INCREMENT	Unique goal identifier
user_id	BIGINT	NO	FK → users.id, INDEX	Owner of the goal

title	VARCHAR(200)	NO	-	Goal name
description	TEXT	YES	-	Optional detailed description
target_date	DATE	YES	-	Target completion date
status	ENUM('ACTIVE','COMPLETED','ABANDONED','PARTIALLY COMPLETED')	NO	DEFAULT 'ACTIVE'	Current status of the goal
created_at	TIMESTAMP	NO	DEFAULT CURRENT_TIMESTAMP	Creation time
updated_at	TIMESTAMP	NO	ON UPDATE CURRENT_TIMESTAMP	Last modification time

goal_habits (Junction Table)

Column	Type	Nullabl e	Constraints	Description
id	BIGINT	NO	PK, AUTO_INCREMENT	Unique record identifier
user_id	BIGINT	NO	FK → users.id, INDEX	Owner of the goal-habit link
goal_id	BIGINT	NO	FK → goals.id, UNIQUE(goal_id, habit_id)	Associated goal
habit_id	BIGINT	NO	FK → habits.id	Associated habit
created_at	TIMESTA MP	NO	DEFAULT CURRENT_TIMESTAMP	Link creation time

Key Indexes

- idx_habits_user_id on habits(user_id) — quick lookup of all habits for a user
- idx_habit_logs_streak on habit_logs(habit_id, logged_date DESC, status) — fast backward scan for consecutive streak counting
- idx_habit_logs_user_id on habit_logs(user_id) — quick lookup of all logs for a user

- `idx_goals_user_id` on `goals(user_id)` — quick lookup of all goals for a user
- `idx_goal_habits_goal_id` on `goal_habits(goal_id)` — join performance
- `idx_goal_habits_user_id` on `goal_habits(user_id)` — quick lookup of all goal-habit links for a user

API Design

The backend exposes a RESTful API with JSON request/response bodies. All endpoints except auth routes require a valid JWT in the `Authorization: Bearer <token>` header.

Authentication Endpoints

Method	Endpoint	Request Body	Response
POST	<code>/api/auth/register</code>	<code>{ email, password, name }</code>	<code>{ user, token }</code> — 201 Created
POST	<code>/api/auth/login</code>	<code>{ email, password }</code>	<code>{ user, token }</code> — 200 OK
GET	<code>/api/auth/me</code>	-	<code>{ user }</code> — 200 OK (requires auth)

Habit Endpoints

Method	Endpoint	Request Body	Response
GET	<code>/api/habits</code>	-	<code>{ habits[] }</code> — list user's habits with streaks
POST	<code>/api/habits</code>	<code>{ title, description?, type }</code>	<code>{ habit }</code> — 201 Created
GET	<code>/api/habits/:id</code>	-	<code>{ habit, logs[] }</code> — habit detail with log history
PUT	<code>/api/habits/:id</code>	<code>{ title?, description?, type? }</code>	<code>{ habit }</code> — 200 Updated
DELETE	<code>/api/habits/:id</code>	-	204 No Content
POST	<code>/api/habits/:id/mark</code>	<code>{ date? }</code> (defaults to today)	<code>{ log, streak }</code> — mark completion/clean day

Goal Endpoints

Method	Endpoint	Request Body	Response
GET	/api/goals	-	{ goals[] } — list user's goals
POST	/api/goals	{ title, description?, target_date?, habit_ids[] }	{ goal } — 201 Created
GET	/api/goals/:id	-	{ goal, habits[] } — with linked habits
PUT	/api/goals/:id	{ title?, description?, target_date?, status?, habit_ids[]? }	{ goal } — 200 Updated
DELETE	/api/goals/:id	-	204 No Content

Streak/Tracking Endpoint

Method	Endpoint	Query Params	Response
GET	/api/habits/:id/stats	from, to (date range)	{ current_streak, longest_streak, total_logs, calendar[] }

Error Response Format

All errors follow a consistent structure:

Code block

```
1  {
2    "error": {
3      "code": "VALIDATION_ERROR",
4      "message": "Human-readable description",
5      "details": [{ "field": "email", "message": "must be a valid email" }]
6    }
7  }
8
```


Frontend Design

Technology Stack

- **React 18** with functional components and hooks
- **TypeScript** for type safety
- **Vite** as build tool (fast HMR, optimized production builds)
- **React Router v6** for client-side routing
- **Axios** for HTTP requests with interceptors
- **React Context + useReducer** for global auth state
- **CSS Modules** or **Tailwind CSS** for styling

Page Structure

Route	Component	Description
/login	LoginPage	Email/password login form
/register	RegisterPage	New account registration
/dashboard	DashboardPage	Overview of today's habits, active streaks, and goals progress
/habits	HabitsListPage	All habits with streak indicators and mark buttons
/habits/:id	HabitDetailPage	Habit history calendar, streak stats, edit form
/goals	GoalsListPage	All goals with linked habits and progress
/goals/:id	GoalDetailPage	Goal details, linked habits, status management

Key Components

- **AuthProvider** — Context provider managing JWT token, user state, login/logout
- **ProtectedRoute** — HOC that redirects unauthenticated users to /login
- **HabitCard** — Reusable card showing habit name, type badge, streak count, and mark action

- **StreakCalendar** — Visual calendar grid showing completion/clean days
- **GoalProgress** — Displays goal with linked habit streak summaries

State Management Strategy

Global State (Context)

- Auth token and user profile
- Loading/error states for auth

Local State (Component)

- Form inputs and validation
- Habit list, goal list (fetched per page)
- Modal/dialog open state

Authentication and Security

Authentication Flow

1. User submits email + password to `/api/auth/login`
2. Server validates credentials against bcrypt hash
3. Server generates a JWT (HS256, 24h expiry) containing `{ userId, email }`
4. Client stores token in `localStorage` and attaches it to all subsequent requests via Axios interceptor
5. Backend middleware extracts and verifies JWT on every protected route
6. On token expiry, client receives 401 and redirects to login

Security Measures



Security Checklist to be considered:

- Passwords hashed with **bcrypt** (cost factor 12)
- JWT secret stored in environment variable, never committed to source
- Input validation via **express-validator** or **zod** on all endpoints
- Rate limiting on auth endpoints (e.g., 5 attempts per minute per IP)
- CORS configured to allow only the frontend origin
- SQL injection prevention via parameterized queries (ORM)
- XSS prevention via React's default escaping + Content-Security-Policy headers

Streak Logic — Lazy Evaluation

Design Approach: Lazy Evaluation

Streaks are calculated **on-demand** when the user accesses the app. This eliminates background infrastructure and timezone batch-processing complexity entirely.

★ **Core Insight:** Store **events** (completions and relapses), not daily status. Derive streaks from events at read time. For breaking habits, the streak is simply `daysBetween(streak_start_date, today)` — no daily log insertion needed.

Code block

```
1 // Core streak calculation – called when user opens app
2 function getCurrentStreak(habit: Habit, user: User): number {
3   const today = getCurrentDate(user.timezone);
4
5   if (habit.type === "BREAKING") {
6     // Simple date math: streak = days since last relapse
7     if (habit.streak_start_date > today) return 0; // relapsed today
8     return daysBetween(habit.streak_start_date, today);
9   } else {
10    // Count consecutive COMPLETED logs backwards
11    return countConsecutiveCompleted(habit.id, today);
12  }
13 }
14
```

Breaking Habits — streak_start_date Approach

For **breaking habits**, the streak is derived from a single stored date — no daily logs needed for calculation:

- `streak_start_date` = day after last relapse (or habit creation date)
- Current streak = `daysBetween(streak_start_date, today)` in user's timezone
- When user relapses: set `streak_start_date = tomorrow`, streak becomes 0
- Absence of relapse IS the clean day — no background job needed

Code block

```
1 // Breaking habit: mark relapse
2 function markBreakingRelapse(user: User, habit: Habit) {
3   const today = getCurrentDate(user.timezone);
```

```

4  const tomorrow = addDays(today, 1);
5
6  // Save longest streak before resetting
7  const currentStreak = daysBetween(habit.streak_start_date, today);
8  if (currentStreak > habit.longest_streak) {
9      updateHabit(habit.id, { longest_streak: currentStreak });
10 }
11
12 // Record relapse event (for history/calendar view)
13 upsertLog(habit.id, user.id, today, "RELAPSED");
14
15 // Reset: streak restarts tomorrow
16 updateHabit(habit.id, { streak_start_date: tomorrow });
17
18 return { currentStreak: 0, longestStreak: habit.longest_streak };
19 }
20

```

Building Habits — Consecutive Log Counting

For **building habits**, the user must actively mark completion. The streak is calculated by counting consecutive **COMPLETED** logs backwards from yesterday:

- User marks **COMPLETED** during the day via the API
- Streak = count consecutive COMPLETED logs backward from yesterday
- If yesterday has no COMPLETED log → streak = 0
- If today is already marked, include today in the count

Code block

```

1  // Building habit: count consecutive completed days
2  function countConsecutiveCompleted(habitId: number, today: string): number {
3      const startFrom = subtractDays(today, 1); // yesterday
4
5      // Single query: get completed logs ordered by date descending
6      const logs = query(`
7          SELECT logged_date FROM habit_logs
8          WHERE habit_id = ? AND logged_date <= ? AND status = "COMPLETED"
9          ORDER BY logged_date DESC
10     `, [habitId, startFrom]);
11
12     let streak = 0;
13     let expectedDate = startFrom;
14
15     for (const log of logs) {

```

```

16     if (log.logged_date === expectedDate) {
17         streak++;
18         expectedDate = subtractDays(expectedDate, 1);
19     } else {
20         break; // Gap found – streak ends here
21     }
22 }
23
24 // Include today if already marked
25 const todayLog = getLog(habitId, today);
26 if (todayLog?.status === "COMPLETED") streak++;
27
28 return streak;
29 }
30
31 // Mark building habit as completed
32 function markBuildingComplete(user: User, habit: Habit) {
33     const today = getCurrentDate(user.timezone);
34     upsertLog(habit.id, user.id, today, "COMPLETED");
35
36     const streak = countConsecutiveCompleted(habit.id, today);
37     if (streak > habit.longest_streak) {
38         updateHabit(habit.id, { longest_streak: streak });
39     }
40     return { currentStreak: streak };
41 }
42

```

Timezone Handling

 **How it works:** Each user stores their IANA timezone (e.g., `Asia/Jakarta`) in the `users` table. When the user accesses the app, `getCurrentDate(user.timezone)` computes “today” and “yesterday” relative to the user’s local time using `Intl.DateTimeFormat`. This means a user in Jakarta (UTC+7) and a user in New York (UTC-4) each see correct streak values — timezone is resolved at request time.

Code block

```

1 function getCurrentDate(timezone: string): string {
2     return new Intl.DateTimeFormat("en-CA", {
3         timeZone: timezone,
4         year: "numeric", month: "2-digit", day: "2-digit"
5     }).format(new Date()); // Returns "YYYY-MM-DD"

```

```
6 }  
7
```

Weekly Streak Dashboard

The weekly dashboard provides a 7-day grid view of all user habits. It requires only **2 database queries** (all habits + all logs for the week) and in-memory mapping:

Code block

```
1 function getWeeklyDashboard(user: User, weekStart: string) {  
2   const today = getCurrentDate(user.timezone);  
3   const weekEnd = addDays(weekStart, 6);  
4  
5   // Query 1: all user habits  
6   const habits = query("SELECT * FROM habits WHERE user_id = ?", [user.id]);  
7  
8   // Query 2: all logs for the week (sparse)  
9   const logs = query(`  
10     SELECT habit_id, logged_date, status FROM habit_logs  
11     WHERE user_id = ? AND logged_date BETWEEN ? AND ?  
12   `, [user.id, weekStart, weekEnd]);  
13  
14   // Build grid: for each habit, derive status per day  
15   return habits.map(habit => ({  
16     habit,  
17     currentStreak: getCurrentStreak(habit, user),  
18     days: buildWeekDays(habit, logs, weekStart, today)  
19   }));  
20 }  
21  
22 function getDayStatus(habit, log, date, today): string {  
23   if (date > today) return "FUTURE";  
24   if (date === today) {  
25     if (habit.type === "BUILDING")  
26       return log?.status === "COMPLETED" ? "COMPLETED" : "PENDING";  
27     else  
28       return log?.status === "RELAPSED" ? "RELAPSED" : "CLEAN_SO_FAR";  
29   }  
30   // Past days  
31   if (habit.type === "BUILDING")  
32     return log?.status === "COMPLETED" ? "COMPLETED" : "MISSED";  
33   else  
34     return log?.status === "RELAPSED" ? "RELAPSED" : "CLEAN";  
35 }
```

API Endpoint: `GET /api/dashboard/weekly?week_start=2026-06-02`

Goal Management

Business Rules

1. Each goal **must be linked to at least 1 habit** at creation time
2. A goal can be linked to multiple habits (many-to-many via `goal_habits`)
3. A habit can belong to multiple goals
4. Goal progress is derived from the combined streak performance of its linked habits

Goal Progress Calculation

Code block

```
1 interface GoalProgress {
2     goal: Goal;
3     linkedHabits: Array<{
4         habit: Habit;
5         currentStreak: number;
6         isOnTrack: boolean; // streak > 0 today
7     }>;
8     overallScore: number; // percentage of linked habits on track
9 }
10
```

Deployment Architecture

Docker Compose Configuration

The application is deployed as a multi-container Docker setup:

Code block

```
1 # docker-compose.yml
2 version: '3.8'
3 services:
4     nginx:
```

```

5  image: nginx:alpine
6  ports: ["80:80", "443:443"]
7  volumes:
8    - ./nginx.conf:/etc/nginx/nginx.conf
9    - ./frontend/dist:/usr/share/nginx/html
10 depends_on: [backend]
11
12 backend:
13   build: ./backend
14   environment:
15     - DB_HOST=mysql
16     - DB_PORT=3306
17     - DB_NAME=habit_shaper
18     - JWT_SECRET=${JWT_SECRET}
19   depends_on:
20     mysql:
21       condition: service_healthy
22
23 mysql:
24   image: mysql:8.0
25   environment:
26     - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
27     - MYSQL_DATABASE=habit_shaper
28   volumes:
29     - mysql_data:/var/lib/mysql
30     - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
31   healthcheck:
32     test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
33     interval: 10s
34     timeout: 5s
35     retries: 5
36
37 volumes:
38   mysql_data:
39

```

Container Responsibilities

Nginx Container

- Serves React SPA static files
- Proxies `/api/*` to backend
- Handles TLS termination

Backend Container

- Node.js + Express app
- Runs database migrations on startup
- Exposes port 3000 internally

MySQL Container

- Persistent data via named volume
- Health check ensures readiness

- Auto-initializes schema on first run

Testing Strategy

Testing Pyramid

Level	Tool	Coverage Target	Examples
Unit Tests	Jest + ts-jest	Business logic, streak calculation, validators	Streak calculator, input validation, JWT utilities
Integration Tests	Supertest + Jest	API endpoint behavior with real DB	Auth flow, CRUD operations, streak updates after marking
Frontend Tests	React Testing Library	Component rendering, user interactions	Login form submission, habit card mark action, route guards
E2E Tests	Cypress / Playwright	Critical user journeys end-to-end	Register → Create habit → Mark → Verify streak → Create goal

Project Structure

Code block

```
1 habit-shaper/
2 |─ docker-compose.yml
3 |─ nginx.conf
4 |─ frontend/
5 |   |─ package.json
6 |   |─ tsconfig.json
7 |   |─ vite.config.ts
8 |   |─ src/
9 |   |   |─ main.tsx
10 |   |   |─ App.tsx
11 |   |   |─ api/           # Axios instance + API calls
12 |   |   |─ components/   # Shared UI components
13 |   |   |─ contexts/     # AuthContext
14 |   |   |─ pages/        # Route-level page components
15 |   |   |─ hooks/        # Custom hooks (useAuth, useHabits)
16 |   |   |─ types/        # TypeScript interfaces
```

```
17 |   └─ Dockerfile
18 |   └─ backend/
19 |       └─ package.json
20 |       └─ tsconfig.json
21 |       └─ src/
22 |           └─ index.ts           # Express app entry
23 |           └─ config/           # Environment config
24 |           └─ middleware/       # Auth, validation, error handler
25 |           └─ routes/           # Route definitions
26 |           └─ controllers/      # Request handlers
27 |           └─ services/         # Business logic (streak calc)
28 |           └─ models/           # ORM models / DB queries
29 |           └─ utils/            # JWT helpers, date utils
30 |   └─ Dockerfile
31 |   └─ db/
32 |       └─ init.sql              # Initial schema DDL
33 |       └─ migrations/          # Incremental migrations
34 |
```

Non-Functional Requirements

Performance

Reliability

- API response time < 200ms (p95)
 - Frontend initial load < 3s on 3G
 - Database queries optimized with proper indexing
 - Connection pooling for MySQL (pool size: 10)
- MySQL data persisted via Docker volumes
 - Graceful shutdown handling in Node.js
 - Health check endpoints for monitoring
 - Structured JSON logging for debugging

Risks and Mitigations

Risk	Impact	Mitigation
Streak calculation becomes expensive at scale	Slow page loads for active users	Cache current streak in habits table; recalculate only on mark action
JWT token theft via XSS	Account compromise	

		CSP headers, HttpOnly cookies (future), short token TTL
MySQL single point of failure	Data loss on container crash	Named volumes with host mount, regular backup cron job
Timezone discrepancies in streak tracking	Incorrect streak counts	Store all dates in UTC; convert to user timezone on frontend